Refined Research Proposal

Context.

Feed Forward Gaussian Splatting (FFGS) are a hot topic
Images => 2D model (VGGT like) => 3DGS

Current limitations.

Limited scalability (# frames)
Bad geometry

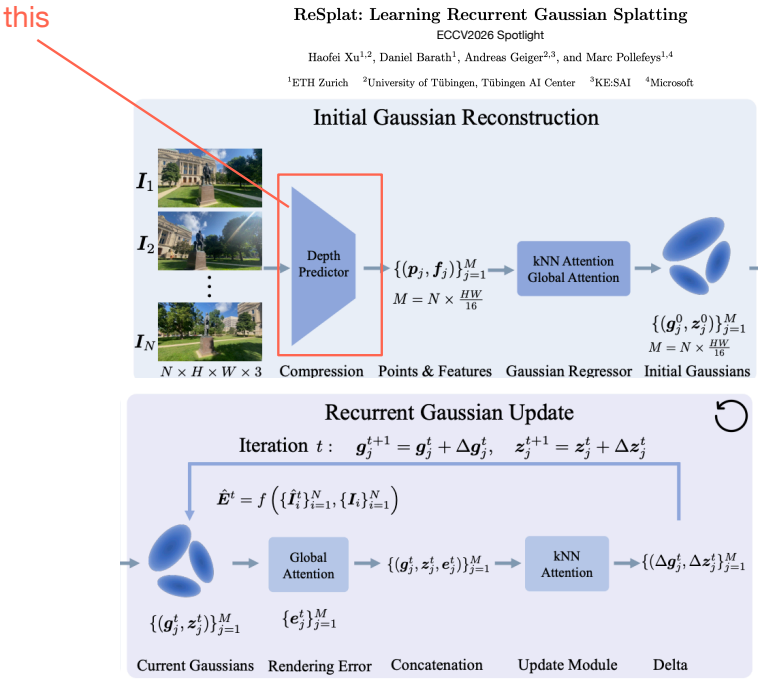
Idea.

Geometry from triangulation has shown great potential (in FF. Recon and GS)
Images => Triangulation => Points => PCL Model => 3DGS

Target.

Show better results (geometrically) on classical datasets
Show better scalability of city scale dataset (Matrix city)

this



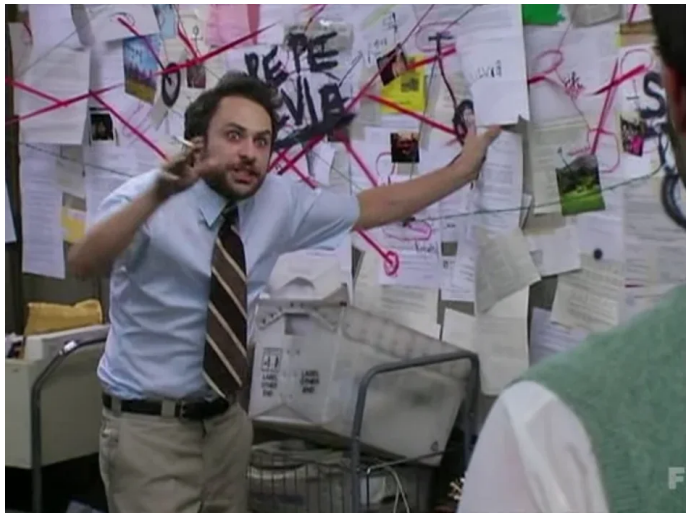
Refined Research Proposal

Context.

Feed Forward Gaussian Splatting (FFGS) are a hot topic
Images => 2D model (VGGT like) => 3DGS

Current limitations.

Limited scalability (# frames)
Bad geometry



Idea.

Geometry from triangulation has shown great potential (in FF. Recon and GS)
Images => Triangulation => Points => PCL Model => 3DGS

Target.

Show better results (geometrically) on classical datasets
Show better scalability of city scale dataset (Matrix city)

Proposed solution.

Proposed solution.

Overview

Images => **Triangulation** => Points => **PT Model** => GS Params

Proposed solution.

Overview

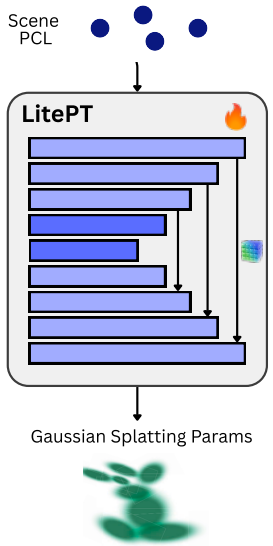
Images => **Triangulation** => Points => **PT Model** => GS Params

Why do we need a Point model?

In GS, texture arises from Gaussian Density, element not properly addressed
AnySplat - averaging; EcoSplat - through 2D

Architectures

Utonia - ICML 26 - self supervised PointTransformer for PCL
LightPT - CVPR 26 - ablated PTv3 version (presented at reading group)



But… how do we get the points?

Proposed solution.

Overview

Images => **Triangulation** => Points => **PT Model** => GS Params

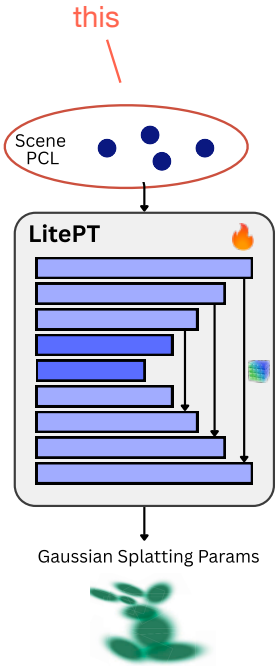
How do we get the points?

ROMAv2 - ECCV ’26 => feature detection & matching

+

EDGS CVPR ’26 => suggested triangulation

or a foundation model such as MVSAnywhere (CVPR ’25) or MAST3R



Proposed solution.

Overview

Images => **Triangulation** => Points => **PT Model** => GS Params

How do we get the points?

ROMAv2 - ECCV ’26 => feature detection & matching

\+

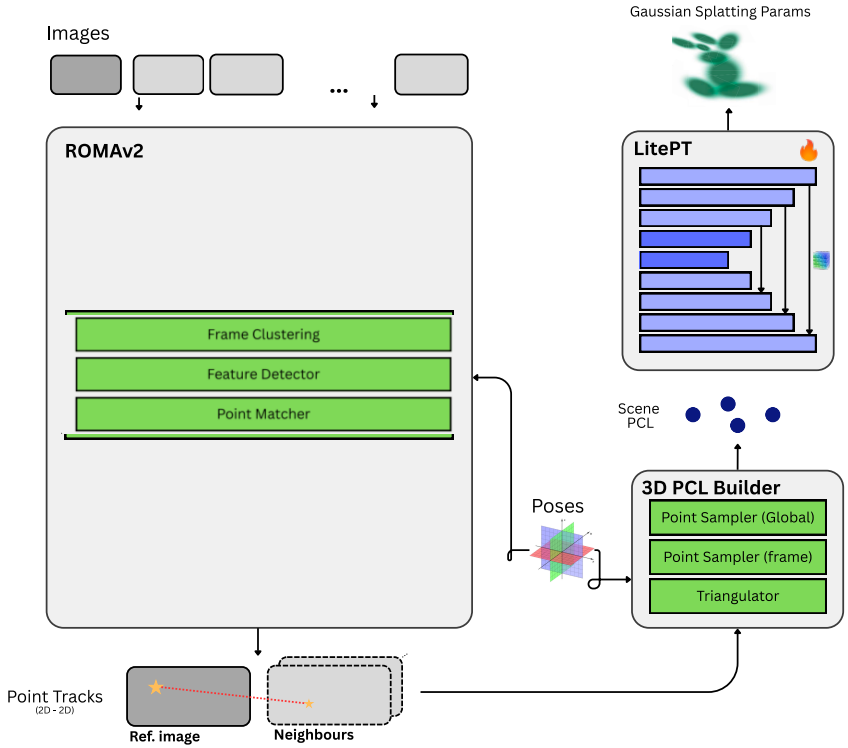
EDGS CVPR ’26 => suggested triangulation

3D PCL Builder

```
all_pts = [ ]

for ref in reference_frames_list:
    pts = triangulate(point_tracks, poses)
    pts = sample_based_on_confidence (pts)
    all_pts.append(pts)

sample_globally (all_pts)
```



Proposed solution.

Overview

Images => **Triangulation** => Points => **PT Model** => GS Params

How do we get the points?

ROMAv2 - ECCV ’26 => feature detection & matching

\+

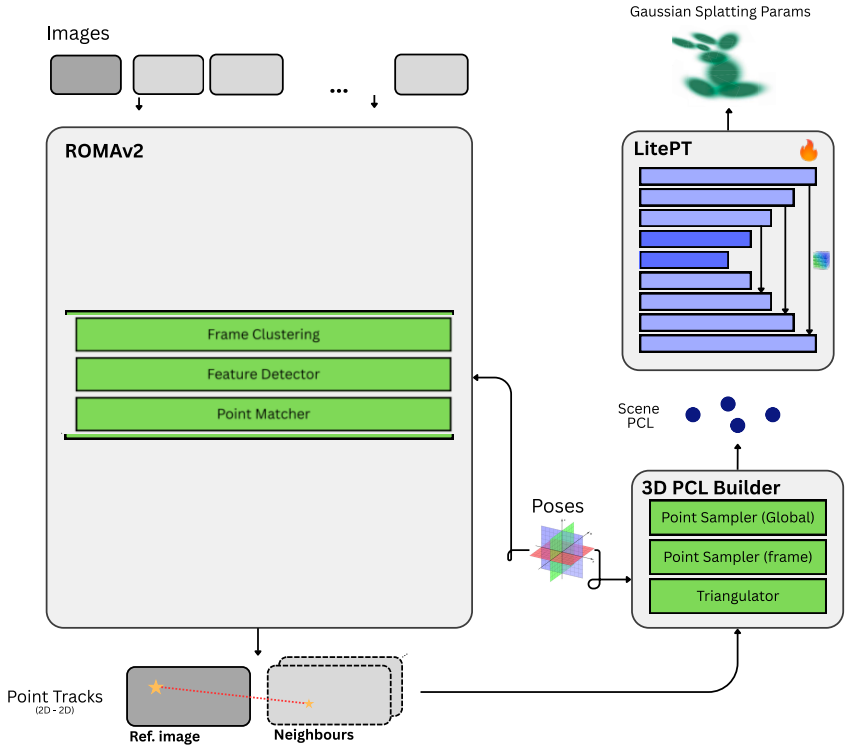
EDGS CVPR ’26 => suggested triangulation

3D PCL Builder

```
all_pts = [ ]

for ref in reference_frames_list:
    pts = triangulate(point_tracks, poses)
    pts = sample_based_on_confidence (pts)
    all_pts.append(pts)

sample_based_on_confidence (all_pts)
```



But… there is no image information. How can we reconstruct it? :/

Proposed solution.

Overview

Images => **Triangulation** => Points => **PT Model** => GS Params

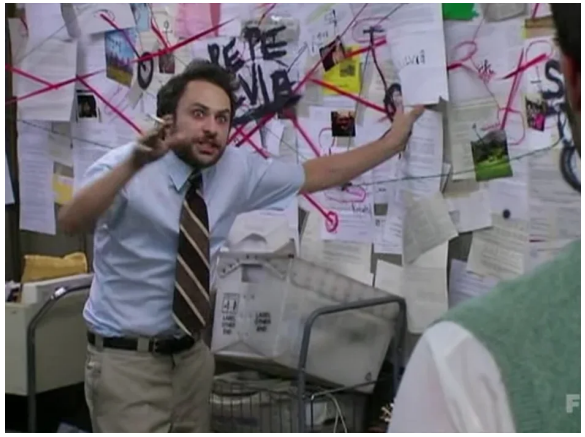
How do we put the image information into points?

Proposed solution.

Overview

Images => **Triangulation** => Points => **PT Model** => GS Params

How do we put the image information into points?



Proposed solution.

Overview

Images => **Triangulation** => Points => **PT Model** => GS Params

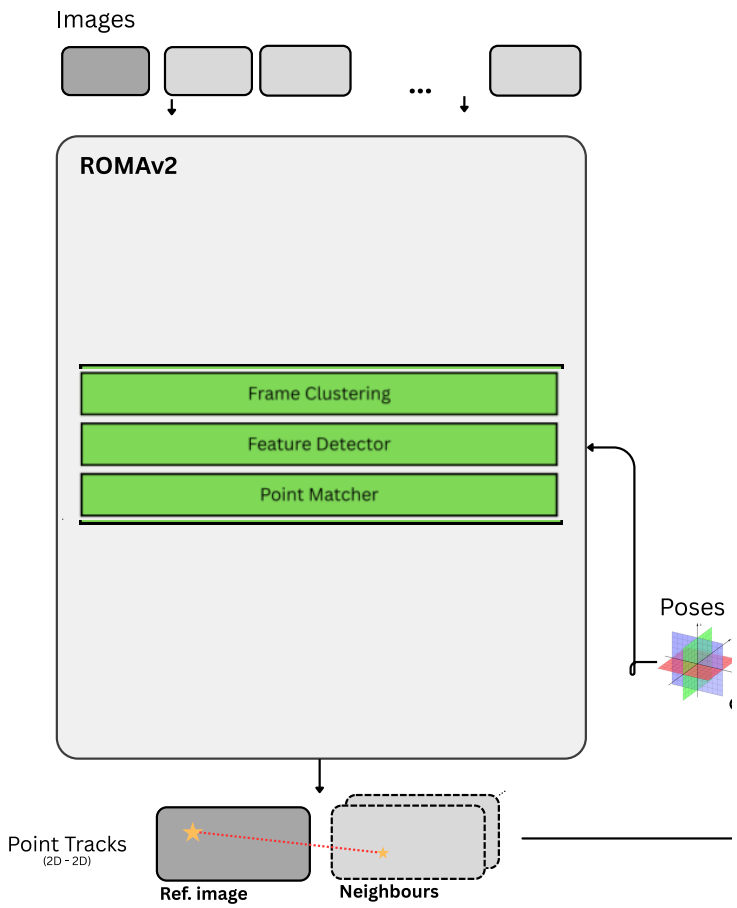
How do we put the image information into points?

Based on ROMAv2

We need global and local texture features

ROMAv2 = VGG + DINOv3

On the outside



Proposed solution.

Overview

Images => **Triangulation** => Points => **PT Model** => GS Params

How do we put the image information into points?

Based on ROMAv2

We need global and local texture features

ROMAv2 = VGG + DINOv3

DINOv3 => global semantic features

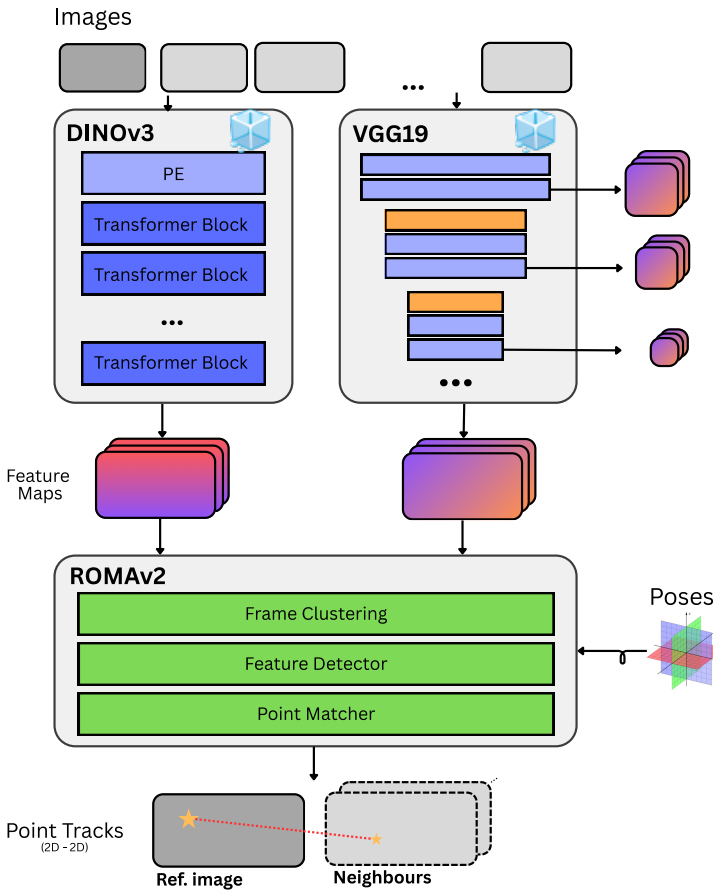
VGG => Local texture features + multi resolution feature maps

But… how do we combine all these features?

DINOv3 - 1024 channels

VGG - 64, 128, 256 channels

On the inside



Proposed solution.

Overview

Images => **Triangulation** => Points => **PT Model** => GS Params

